

**PRAKTIKUM SISTEM CERDAS DAN PENDUKUNG KEPUTUSAN
SEMESTER GENAP TA 2025/2026**

LAPORAN PROYEK AKHIR



DISUSUN OLEH:

NIM	:	123240005 123240020
NAMA	:	Muhammad Hasan Rabbani Ahmadsyah Anshori Melayu
PLUG	:	IF-E
NAMA ASISTEN	:	Khatama Putra Bintoro

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2026**

HALAMAN PENGESAHAN

LAPORAN PROYEK AKHIR

Disusun oleh:

Muhammad Hasan Rabbani 123240005

Ahmadsyah Anshori 123240020

Telah Diperiksa dan Disetujui oleh Asisten Praktikum Sistem Cerdas dan Pendukung
Keputusan

Pada Tanggal:

Asisten Praktikum

Asisten Praktikum

Khatama Putra
NIM. 123230053

Bintoro
NIM. 123230059

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa yang senantiasa mencurahkan rahmat dan hidayah-Nya sehingga saya dapat menyelesaikan praktikum Sistem Cerdas dan Pendukung Keputusan serta laporan proyek akhir praktikum yang berjudul Jogja Travel Recommendation. Adapun laporan ini berisi tentang proyek akhir yang saya pilih dari hasil pembelajaran selama praktikum berlangsung.

Tidak lupa ucapan terimakasih kepada asisten praktikum yang selalu membimbing dan mengajari saya dalam melaksanakan praktikum dan dalam menyusun laporan ini. Laporan ini masih sangat jauh dari kesempurnaan, oleh karena itu kritik serta saran yang membangun saya harapkan untuk menyempurnakan laporan akhir ini.

Atas perhatian dari semua pihak yang membantu penulisan ini, saya ucapkan terimakasih. Semoga laporan ini dapat dipergunakan seperlunya.

Yogyakarta, 18 Mei 2026

Penyusun 1,

Penyusun 2,

Muhammad Hasan Rabbani

NIM. 123240005

Ahmadsyah Anshori

NIM. 123240020

DAFTAR ISI

HALAMAN PENGESAHAN	ii
KATA PENGANTAR	iii
DAFTAR ISI.....	iv
BAB I PENDAHULUAN	1
1.1 Latar Belakang Masalah.....	1
1.2 Tujuan Proyek Akhir.....	1
1.3 Manfaat Proyek Akhir.....	2
BAB II PEMBAHASAN	3
2.1 Dasar Teori.....	3
2.2 Skenario Kasus & Dataset.....	3
2.3 Pemodelan Sistem Pendukung Keputusan	4
2.4 Perhitungan & Algoritma.....	5
2.5 Implementasi & Tampilan Sistem.....	6
BAB III JADWAL Pengerjaan DAN PEMBAGIAN TUGAS.....	20
3.1 Jadwal Pengerjaan.....	20
3.2 Pembagian Tugas	21
BAB IV KESIMPULAN DAN SARAN	22
4.2 Kesimpulan	22
4.3 Saran.....	22
DAFTAR PUSTAKA	23

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Perkembangan sektor pariwisata di Yogyakarta menyebabkan jumlah destinasi wisata terus meningkat dari tahun ke tahun. Wisatawan kini memiliki banyak pilihan tempat wisata, mulai dari wisata alam, budaya, pantai, museum, hingga wisata buatan. Banyaknya pilihan tersebut sering kali membuat wisatawan kesulitan menentukan destinasi yang paling sesuai dengan kebutuhan dan preferensi mereka.

Dalam memilih tempat wisata, terdapat beberapa faktor yang perlu dipertimbangkan, seperti rating wisatawan, harga tiket masuk, fasilitas, aksesibilitas, dan popularitas destinasi. Jika proses pemilihan dilakukan secara manual, maka akan membutuhkan waktu yang cukup lama dan hasilnya belum tentu objektif.

Oleh karena itu, dibutuhkan sebuah Sistem Pendukung Keputusan (SPK) yang dapat membantu pengguna menentukan rekomendasi destinasi wisata terbaik secara lebih cepat, efektif, dan terstruktur. Pada program ini digunakan metode Simple Additive Weighting (SAW), yaitu metode pengambilan keputusan multikriteria yang bekerja dengan cara memberikan bobot pada setiap kriteria, melakukan normalisasi data, kemudian menghitung nilai akhir untuk menentukan peringkat terbaik.

Aplikasi ini dibangun menggunakan bahasa pemrograman Python dengan framework Streamlit sehingga menghasilkan tampilan interaktif dan mudah digunakan oleh pengguna.

1.2 Tujuan Proyek Akhir

1. Membuat sistem Pendukung Keputusan (SPK) untuk rekomendasi destinasi wisata di Yogyakarta.
2. Mengimplementasikan metode Simple Additive Weighting (SAW) untuk proses perangkingan destinasi wisata.
3. Membantu pengguna memilih destinasi wisata berdasarkan kriteria tertentu seperti rating, harga tiket, fasilitas, aksesibilitas, dan popularitas.
4. Menyediakan visualisasi data wisata agar pengguna lebih mudah memahami informasi yang tersedia.
5. Membuat aplikasi berbasis web interaktif menggunakan Streamlit agar mudah digunakan.

1.3 Manfaat Proyek Akhir

1. Bagi Pengguna / Wisatawan

- Membantu wisatawan memilih destinasi wisata terbaik sesuai preferensi.
- Menghemat waktu dalam proses pencarian tempat wisata.
- Memberikan rekomendasi yang lebih objektif berdasarkan data dan perhitungan.

2. Bagi Pengembang / Peneliti

- Menjadi media pembelajaran implementasi metode SAW dalam Sistem Pendukung Keputusan.
- Menambah pengalaman dalam pengolahan data menggunakan Python, Pandas, NumPy, dan Streamlit.
- Menjadi contoh penerapan data mining dan SPK dalam bidang pariwisata.

3. Bagi Dunia Akademik

- Dapat dijadikan referensi penelitian terkait Sistem Pendukung Keputusan.
- Menjadi contoh implementasi metode pengambilan keputusan multikriteria pada studi kasus nyata.

BAB II PEMBAHASAN

2.1 Dasar Teori

Sistem Pendukung Keputusan (SPK) merupakan sistem berbasis komputer yang digunakan untuk membantu pengambilan keputusan dalam suatu masalah yang melibatkan banyak kriteria. Pada penelitian ini digunakan metode Simple Additive Weighting (SAW), yaitu salah satu metode dalam pengambilan keputusan multikriteria yang paling sederhana dan banyak digunakan.

Metode SAW bekerja dengan cara menjumlahkan nilai terbobot dari setiap alternatif pada semua kriteria. Setiap kriteria memiliki bobot yang menunjukkan tingkat kepentingannya. Sebelum dilakukan perhitungan, data harus dinormalisasi terlebih dahulu agar memiliki skala yang sama.

Terdapat dua jenis kriteria dalam metode SAW, yaitu:

- **Benefit** → semakin besar nilainya semakin baik
- **Cost** → semakin kecil nilainya semakin baik

Rumus normalisasi:

- **Benefit:**
$$rij = x_{ij} / \max(x_{ij})$$
- **Cost:**
$$rij = \min(x_{ij}) / x_{ij}$$

Rumus nilai preferensi:

$$V_i = \sum (w_j \times rij)$$

Keterangan:

- V_i = nilai akhir alternatif
- w_j = bobot kriteria
- rij = nilai hasil normalisasi

Alternatif dengan nilai V_i terbesar akan menjadi pilihan terbaik.

2.2 Skenario Kasus & Dataset

Skenario Kasus

Permasalahan yang diangkat adalah bagaimana membantu wisatawan dalam memilih destinasi wisata terbaik di Yogyakarta. Banyaknya pilihan destinasi membuat proses pemilihan menjadi sulit, terutama jika mempertimbangkan beberapa faktor sekaligus.

Dalam sistem ini:

- **Pengambil keputusan:** Wisatawan / pengguna aplikasi
- **Tujuan akhir:** Menentukan destinasi wisata terbaik berdasarkan preferensi pengguna
- **Pendekatan:** Menggunakan metode SAW untuk melakukan perangkingan

Pengguna dapat menentukan prioritas kriteria melalui bobot yang diinputkan, sehingga hasil rekomendasi dapat disesuaikan dengan kebutuhan masing-masing.

Dataset

Dataset yang digunakan adalah data destinasi wisata Yogyakarta yang berjumlah ±437 data. Data ini berisi informasi seperti:

- Nama destinasi
- Jenis wisata
- Rating
- Harga tiket masuk (HTM)

Selain itu, sistem juga menambahkan atribut turunan, yaitu:

- Fasilitas (berdasarkan jenis wisata)
- Aksesibilitas (dibangkitkan secara acak)
- Popularitas (berdasarkan rating)

Sumber dataset: Kaggle (Tourism Jogja Dataset)

Contoh data sederhana:

Alternatif	Rating	HTM	Fasilitas	Akses	Popularitas
A1	4.5	10000	4	3	4
A2	4.2	5000	3	4	3
A3	4.7	15000	5	5	5

2.3 Pemodelan Sistem Pendukung Keputusan

Kriteria:

Kode	Kriteria	Jenis
C1	Rating	Benefit
C2	Harga Tiket	Cost
C3	Fasilitas	Benefit
C4	Aksesibilitas	Benefit
C5	Popularitas	Benefit

Alternatif:

Dalam sistem ini adalah destinasi wisata yang tersedia dalam dataset, contoh:

- A1: Pantai Parangtritis
- A2: Malioboro
- A3: Candi Prambanan

Skala Penilaian:

Setiap kriteria memiliki skala sebagai berikut:

- Rating: 1 – 5
- Harga Tiket: nilai rupiah
- Fasilitas: 1 – 5
- Aksesibilitas: 1 – 5
- Popularitas: 1 – 5

Bobot Kriteria (contoh):

Kriteria	Bobot
C1	0.30
C2	0.25
C3	0.20
C4	0.15
C5	0.10

2.4 Perhitungan & Algoritma

Langkah-langkah metode SAW dalam sistem ini adalah:

1. Menentukan Kriteria dan Bobot

Pengguna menentukan tingkat kepentingan setiap kriteria dalam bentuk bobot.

2. Menyusun Matriks Keputusan

Matriks berisi nilai setiap alternatif terhadap semua nilai kriteria

Contoh:

Alternatif	C1	C2	C3	C4	C5
A1	4.5	10000	4	3	4
A2	4.2	5000	3	4	3
A3	4.7	15000	5	5	5

3. Normalisasi Matriks

- Benefit -> dibagi nilai maksimum
- Cost -> nilai minimum dibagi nilai

Hasil normalisasi akan menghasilkan nilai antara 0-1.

4. Perhitungan Nilai Preferensi

Setiap nilai normalisasi dikalikan dengan bobot, kemudian dijumlahkan:

$$V_i = (w_1 \times r_1) + (w_2 \times r_2) + \dots + (w_n \times r_n)$$

5. Perankingan

- Nilai akhir diurutkan dari terbesar ke terkecil.

- Alternatif dengan nilai tertinggi menjadi rekomendasi utama.

2.5 Implementasi & Tampilan Sistem

```
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")

# -----
# PAGE CONFIG
# -----
st.set_page_config(
    page_title="SPK Wisata Jogja - Metode SAW",
    layout="wide",
)

# -----
# GLOBAL STYLE
# -----
st.markdown("""
<style>
    .main-title { font-size:2rem; font-weight:700; color:#1a3a5c;
margin-bottom:0; }
    .sub-title { font-size:1rem; color:#5a7a9c; margin-top:0; }
    .metric-box { background:#f0f6ff; border-left:4px solid #2563eb;
padding:12px 16px; border-radius:8px; margin:6px 0; }
    .saw-step { background:#f8fafc; border:1px solid #e2e8f0;
border-radius:8px; padding:14px; margin:8px 0; }
</style>
""", unsafe_allow_html=True)

# -----
# LOAD & PREPARE DATA
# -----
@st.cache_data
def load_data():
    df = pd.read_csv("tourism_jogja.csv")

    # C3: Skor Fasilitas - diturunkan dari jenis wisata
    fasilitas_map = {
        "Buatan": 5, "Wisata Air": 4, "Pantai": 4,
        "Alam": 3, "Budaya Dan Sejarah": 3, "Agrowisata": 3,
        "Museum": 3, "Desa Wisata": 2, "Religi": 2,
        "Kuliner": 2, "Pendidikan": 2, "Seni": 1, "Minat Khusus": 1
    }

    df["fasilitas"] =
df["type"].map(fasilitas_map).fillna(2).astype(int)

    # C4: Skor Aksesibilitas (1-5) - seed deterministik per place_id
    rng = np.random.default_rng(seed=42)
    df["aksesibilitas"] = rng.integers(2, 6, size=len(df))

    # C5: Skor Popularitas - normalisasi berbasis rating
```

```

        df["popularitas"] = (
            pd.cut(df["rating"], bins=[0, 3.9, 4.2, 4.5, 4.7, 5.0],
                labels=[1, 2, 3, 4, 5]).astype(float)
        ).fillna(3).astype(int)

    return df

df_raw = load_data()

# -----
# KRITERIA DEFINITION
# -----
KRITERIA = {
    "C1": {"label": "Rating Wisatawan", "col": "rating", "tipe":
"benefit"},
    "C2": {"label": "Harga Tiket (HTM)", "col": "htm", "tipe":
"cost"},
    "C3": {"label": "Fasilitas", "col": "fasilitas", "tipe":
"benefit"},
    "C4": {"label": "Aksesibilitas", "col": "aksesibilitas", "tipe":
"benefit"},
    "C5": {"label": "Popularitas", "col": "popularitas", "tipe":
"benefit"},
}

# -----
# SIDEBAR NAVIGATION
# -----
with st.sidebar:
    st.markdown("## SPK Wisata Jogja")
    st.markdown("***Metode:** Simple Additive Weighting (SAW)")
    st.markdown("---")
    halaman = st.radio(
        "Navigasi Halaman",
        ["Beranda", "Dataset", "Hitung SPK", "Visualisasi"],
    )
    st.markdown("---")
    st.caption("Dataset: Tourism Jogja (437 destinasi)")
    st.caption("Sumber: Kaggle / Data Primer")

# =====
# HALAMAN 1 - BERANDA
# =====
if halaman == "Beranda":
    st.markdown('<p class="main-title">Sistem Pendukung Keputusan</p>',
        unsafe_allow_html=True)
    st.markdown('<p class="sub-title">Rekomendasi Destinasi Wisata
        Yogyakarta - Metode SAW</p>', unsafe_allow_html=True)
    st.markdown("---")

    col1, col2, col3, col4 = st.columns(4)
    col1.metric("Total Destinasi", f"{len(df_raw):,}")
    col2.metric("Jenis Wisata", df_raw["type"].nunique())
    col3.metric("Rata-rata Rating", f"{df_raw['rating'].mean():.2f}")
    col4.metric("Rata-rata HTM", f"Rp {df_raw['htm'].mean():,.0f}")

    st.markdown("---")
    st.markdown("### Tentang Sistem Ini")
    st.info(

```

```

        "Sistem ini membantu wisatawan memilih destinasi terbaik di
        Yogyakarta "
        "berdasarkan 5 kriteria menggunakan metode Simple Additive
        Weighting (SAW). "
        "User dapat menyesuaikan bobot kriteria sesuai preferensi
        pribadi."
    )

    st.markdown("### Kriteria Penilaian")
    df_kriteria = pd.DataFrame([
        {"Kode": k, "Kriteria": v["label"], "Tipe":
v["tipe"].capitalize(),
        "Kolom Data": v["col"]}]
        for k, v in KRITERIA.items()
    ])
    st.dataframe(df_kriteria, use_container_width=True, hide_index=True)

    st.markdown("### Alur Metode SAW")
    col_a, col_b, col_c, col_d = st.columns(4)
    col_a.markdown("**1. Input Data**\n\nDataset 437 destinasi wisata
Jogja")
    col_b.markdown("**2. Normalisasi**\n\nMatriks ternormalisasi
(benefit/cost)")
    col_c.markdown("**3. Pembobotan**\n\nSkor = Jumlah(bobot x nilai
ternormalisasi)")
    col_d.markdown("**4. Perangkingan**\n\nUrut dari skor tertinggi")

# =====
# HALAMAN 2 - DATASET
# =====

elif halaman == "Dataset":
    st.title("Dataset Destinasi Wisata Jogja")
    st.markdown(f"Total **{len(df_raw)} baris** data |
**{len(df_raw.columns)} kolom**")

    col1, col2 = st.columns([1, 3])
    with col1:
        filter_tipe = st.multiselect(
            "Filter Jenis Wisata",
            options=sorted(df_raw["type"].unique()),
            default=[],
            placeholder="Semua jenis",
        )
        min_rating = st.slider("Minimum Rating", 2.5, 5.0, 4.0, 0.1)

    df_view = df_raw.copy()
    if filter_tipe:
        df_view = df_view[df_view["type"].isin(filter_tipe)]
        df_view = df_view[df_view["rating"] >= min_rating]

    with col2:
        st.markdown(f"Menampilkan **{len(df_view)} destinasi** sesuai
filter")
        st.dataframe(
            df_view[["place_id", "name", "type", "rating", "htm",
                    "fasilitas", "aksesibilitas",
                    "popularitas"]].rename(columns={
                "place_id": "ID", "name": "Nama Destinasi", "type":
                "Jenis",

```

```

        "rating": "Rating", "htm": "HTM (Rp)",
        "fasilitas": "Fasilitas", "aksesibilitas":
"Aksesibilitas",
        "popularitas": "Popularitas"
    }),
    use_container_width=True,
    height=460,
)

st.markdown("---")
st.markdown("### Statistik Deskriptif")
st.dataframe(
    df_raw[["rating", "htm", "fasilitas", "aksesibilitas",
"popularitas"]]
    .describe().round(2),
    use_container_width=True
)

# =====
# HALAMAN 3 - HITUNG SPK
# =====
elif halaman == "Hitung SPK":
    st.title("Perhitungan SPK - Metode SAW")

    # Panel Input Bobot
    st.markdown("### Input Bobot Kriteria")
    st.markdown("Atur bobot setiap kriteria (total harus = 100%). Bobot
mencerminkan prioritas kamu.")

    col1, col2, col3, col4, col5 = st.columns(5)
    w1 = col1.slider("C1 - Rating", 0, 100, 30, 5, help="Bobot
Rating Wisatawan")
    w2 = col2.slider("C2 - HTM", 0, 100, 25, 5, help="Bobot
Harga Tiket")
    w3 = col3.slider("C3 - Fasilitas", 0, 100, 20, 5, help="Bobot
Fasilitas")
    w4 = col4.slider("C4 - Aksesibilitas", 0, 100, 15, 5, help="Bobot
Aksesibilitas")
    w5 = col5.slider("C5 - Popularitas", 0, 100, 10, 5, help="Bobot
Popularitas")

    total_bobot = w1 + w2 + w3 + w4 + w5
    if total_bobot != 100:
        st.warning(f"Total bobot saat ini = **{total_bobot}** (harus
tepat 100 agar valid)")
    else:
        st.success(f"Total bobot = {total_bobot} - Valid!")

    # Filter Jenis Wisata
    st.markdown("### Filter Preferensi")
    col_f1, col_f2, col_f3 = st.columns(3)
    with col_f1:
        filter_jenis = st.multiselect(
            "Jenis Wisata yang Diminati",
            options=sorted(df_raw["type"].unique()),
            default=["Alam", "Budaya Dan Sejarah", "Pantai"],
        )
    with col_f2:
        max_htm = st.number_input(

```

```

        "Batas Maksimal HTM (Rp)", min_value=0, max_value=1_500_000,
        value=100_000, step=5_000,
    )
    with col_f3:
        top_n = st.selectbox("Tampilkan Top N Hasil", [10, 20, 30, 50,
100], index=1)

        st.markdown("---")
        hitung = st.button("Jalankan Perhitungan SAW", type="primary",
use_container_width=True)

        if hitung:
            if total_bobot != 100:
                st.error("Harap sesuaikan bobot hingga totalnya tepat 100
sebelum menghitung.")
            else:
                df_calc = df_raw.copy()
                if filter_jenis:
                    df_calc = df_calc[df_calc["type"].isin(filter_jenis)]
                    df_calc = df_calc[df_calc["htm"] <=
max_htm].reset_index(drop=True)

                if len(df_calc) < 2:
                    st.error("Data terlalu sedikit setelah filter. Longgarkan
filter terlebih dahulu.")
                else:
                    bobot = np.array([w1, w2, w3, w4, w5]) / 100
                    cols = [v["col"] for v in KRITERIA.values()]
                    tipes = [v["tipe"] for v in KRITERIA.values()]

                    matriks = df_calc[cols].values.astype(float)

                    # STEP 1: Tampilkan Matriks Keputusan
                    with st.expander("STEP 1 - Matriks Keputusan (10 baris
pertama)", expanded=True):
                        df_matriks = pd.DataFrame(
                            matriks[:10],
                            columns=[v["label"] for v in KRITERIA.values()],
                            index=df_calc["name"].iloc[:10]
                        ).round(4)
                        st.dataframe(df_matriks, use_container_width=True)

                    # STEP 2: Normalisasi SAW
                    norm = np.zeros_like(matriks)
                    for j, tipe in enumerate(tipes):
                        col_vals = matriks[:, j]
                        if tipe == "benefit":
                            denom = col_vals.max()
                            norm[:, j] = col_vals / denom if denom != 0 else
0
                        else: # cost
                            nonzero = col_vals[col_vals > 0]
                            if len(nonzero) > 0:
                                denom = nonzero.min()
                                norm[:, j] = np.where(col_vals == 0, 1.0,
denom / col_vals)
                            else:
                                norm[:, j] = 1.0

```

```

with st.expander("STEP 2 - Matriks Normalisasi SAW (10
baris pertama)"):
    df_norm = pd.DataFrame(
        norm[:10],
        columns=[v["label"] for v in KRITERIA.values()],
        index=df_calc["name"].iloc[:10]
    ).round(4)
    st.dataframe(df_norm, use_container_width=True)

# STEP 3: Pembobotan & Skor Akhir
skor = np.dot(norm, bobot)
df_calc["Skor SAW"] = np.round(skor, 4)
df_result = (
    df_calc[["name", "type", "rating", "htm",
            "fasilitas", "aksesibilitas", "popularitas",
"Skor SAW"]]
    .sort_values("Skor SAW", ascending=False)
    .head(top_n)
    .reset_index(drop=True)
)
df_result.index += 1
df_result.index.name = "Peringkat"

with st.expander("STEP 3 - Bobot yang Digunakan"):
    df_bobot = pd.DataFrame({
        "Kriteria": [v["label"] for v in
KRITERIA.values()],
        "Tipe": [v["tipe"].capitalize() for v in
KRITERIA.values()],
        "Bobot": bobot,
        "Bobot (%)": [f"{b*100:.0f}%" for b in bobot],
    })
    st.dataframe(df_bobot, use_container_width=True,
hide_index=True)

# STEP 4: Tabel Hasil Perangkingan
st.markdown("---")
st.markdown("### Hasil Perangkingan Destinasi Wisata")
st.markdown(
    f"Menampilkan **Top {top_n}** dari **{len(df_calc)}
destinasi** "
    f"yang sesuai filter."
)

def highlight_rows(row):
    if row.name == 1:
        return ["background-color: #218517"] * len(row)
    elif row.name == 2:
        return ["background-color: #2B9E20"] * len(row)
    elif row.name == 3:
        return ["background-color: #45BF39"] * len(row)
    return [""] * len(row)

df_display = df_result.rename(columns={
    "name": "Nama Destinasi", "type": "Jenis Wisata",
    "rating": "Rating", "htm": "HTM (Rp)",
    "fasilitas": "Fasilitas", "aksesibilitas":
"Aksesibilitas",
    "popularitas": "Popularitas",

```



```

# Grafik 2: Pie Chart Rating Rata-rata per Jenis Wisata
st.markdown("### 2. Perbandingan Rating per Jenis Wisata (Pie Chart)")
top_types = type_counts[type_counts >= 10].index.tolist()
df_pie2 = df_raw[df_raw["type"].isin(top_types)]
avg_rating_per_type = df_pie2.groupby("type")["rating"].mean().reindex(top_types)

fig2, ax2 = plt.subplots(figsize=(9, 9))
colors_pie2 = plt.cm.Set2(np.linspace(0, 1, len(top_types)))
avg_vals = avg_rating_per_type.values
labels_with_rating = [
    f"{t}\n(avg: {r:.2f})" for t, r in zip(avg_rating_per_type.index,
avg_vals)
]
wedges, texts, autotexts = ax2.pie(
    avg_vals,
    labels=labels_with_rating,
    autopct="%1.1f%%",
    colors=colors_pie2,
    startangle=140,
    pctdistance=0.75,
    wedgeprops=dict(edgecolor="white", linewidth=1.2),
)
for text in texts:
    text.set_fontsize(8.5)
for autotext in autotexts:
    autotext.set_fontsize(7.5)
    autotext.set_color("white")
    autotext.set_fontweight("bold")
ax2.set_title("Proporsi Rata-rata Rating per Jenis Wisata",
fontsize=13, fontweight="bold", pad=20)
plt.tight_layout()
st.pyplot(fig2)
plt.close()

# Grafik 3: Scatter HTM vs Rating
st.markdown("### 3. Hubungan Harga Tiket vs Rating Wisatawan")
df_scatter = df_raw[df_raw["htm"] <= 200_000].copy()
jenis_list = df_raw["type"].value_counts().head(6).index.tolist()
df_scatter = df_scatter[df_scatter["type"].isin(jenis_list)]

fig3, ax3 = plt.subplots(figsize=(10, 5))
colors_sc = plt.cm.tab10(np.linspace(0, 1, len(jenis_list)))
for i, jenis in enumerate(jenis_list):
    sub = df_scatter[df_scatter["type"] == jenis]
    ax3.scatter(sub["htm"], sub["rating"], label=jenis,
                color=colors_sc[i], alpha=0.65, s=50,
edgecolors="white", linewidths=0.4)

ax3.set_xlabel("Harga Tiket / HTM (Rp)", fontsize=11)
ax3.set_ylabel("Rating Wisatawan", fontsize=11)
ax3.set_title("Scatter: Harga Tiket vs Rating (HTM <= Rp 200.000)",
fontsize=13, fontweight="bold")
ax3.legend(title="Jenis Wisata", fontsize=8, title_fontsize=9,
bbox_to_anchor=(1.01, 1))
ax3.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f"Rp
{int(x):,}"))
ax3.spines[["top", "right"]].set_visible(False)

```

```

plt.tight_layout()
st.pyplot(fig3)
plt.close()

# Grafik 4: Histogram Rating
st.markdown("### 4. Distribusi Rating Seluruh Destinasi")
fig4, ax4 = plt.subplots(figsize=(8, 4))
ax4.hist(df_raw["rating"], bins=20, color="#3b82f6",
edgecolor="white", alpha=0.85)
ax4.axvline(df_raw["rating"].mean(), color="#ef4444", linestyle="--",
",
linewidth=2, label=f"Rata-rata:
{df_raw['rating'].mean():.2f}")
ax4.set_xlabel("Rating", fontsize=11)
ax4.set_ylabel("Jumlah Destinasi", fontsize=11)
ax4.set_title("Distribusi Rating Wisatawan - 437 Destinasi",
fontsize=13, fontweight="bold")
ax4.legend(fontsize=10)
ax4.spines[["top", "right"]].set_visible(False)
plt.tight_layout()
st.pyplot(fig4)
plt.close()

# Grafik 5: Top 10 Skor SAW (jika sudah dihitung)
if "df_result" in st.session_state:
    st.markdown("### 5. Top 10 Destinasi - Skor SAW")
    top10 = st.session_state["df_result"].head(10)
    colors_saw = plt.cm.YlOrRd_r(np.linspace(0.2, 0.9, len(top10)))
    fig5, ax5 = plt.subplots(figsize=(10, 5))
    bars = ax5.barh(top10["name"], top10["Skor SAW"],
color=colors_saw)
    ax5.bar_label(bars, fmt="%.4f", padding=3, fontsize=9)
    ax5.set_xlabel("Skor SAW", fontsize=11)
    ax5.set_title("Top 10 Destinasi Wisata Berdasarkan Skor SAW",
fontsize=13, fontweight="bold")
    ax5.invert_yaxis()
    ax5.spines[["top", "right"]].set_visible(False)
    plt.tight_layout()
    st.pyplot(fig5)
    plt.close()
else:
    st.info("Jalankan perhitungan SAW di halaman Hitung SPK terlebih
dahulu untuk melihat grafik skor SAW.")

```

SPK Wisata Jogja - Metode SAW

localhost:8501

Deploy

SPK Wisata Jogja

Metode: Simple Additive Weighting (SAW)

Navigasi Halaman

Beranda

Dataset

Hitung SPK

Visualisasi

Dataset: Tourism Jogja (437 destinasi)

Sumber: Kaggle / Data Primer

Sistem Pendukung Keputusan

Rekomendasi Destinasi Wisata Yogyakarta - Metode SAW

Total Destinasi

Jenis Wisata

Rata-rata Rating

Rata-rata HTM

437

13

4.44

Rp 21,820

Tentang Sistem Ini

Sistem ini membantu wisatawan memilih destinasi terbaik di Yogyakarta berdasarkan 5 kriteria menggunakan metode Simple Additive Weighting (SAW). User dapat menyesuaikan bobot kriteria sesuai preferensi pribadi.

Kriteria Penilaian

Kode	Kriteria	Tipe	Kolom Data
C1	Rating Wisatawan	Benefit	rating
C2	Harga Tiket (HTM)	Cost	htm
C3	Fasilitas	Benefit	fasilitas
C4	Aksesibilitas	Benefit	aksesibilitas
C5	Popularitas	Benefit	popularitas

Alur Metode SAW

1. Input Data

Dataset 437 destinasi wisata Jogja

2. Normalisasi

Matriks ternormalisasi (benefit/cost)

3. Pembobotan

Skor = Jumlah(bobot x nilai ternormalisasi)

4. Perangkingan

Urut dari skor tertinggi

SPK Wisata Jogja

Metode: Simple Additive Weighting (SAW)

Navigasi Halaman

Beranda

Dataset

Hitung SPK

Visualisasi

Dataset: Tourism Jogja (437 destinasi)

Sumber: Kaggle / Data Primer

Dataset Destinasi Wisata Jogja

Total 437 baris data | 11 kolom

Filter Jenis Wisata

Semua jenis

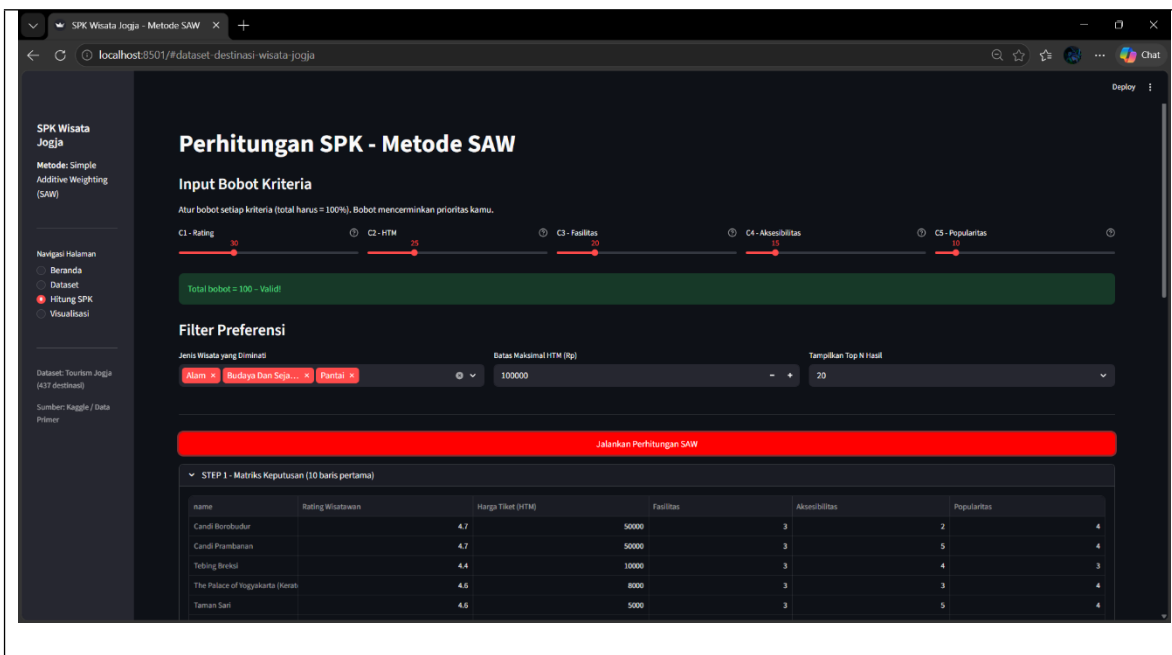
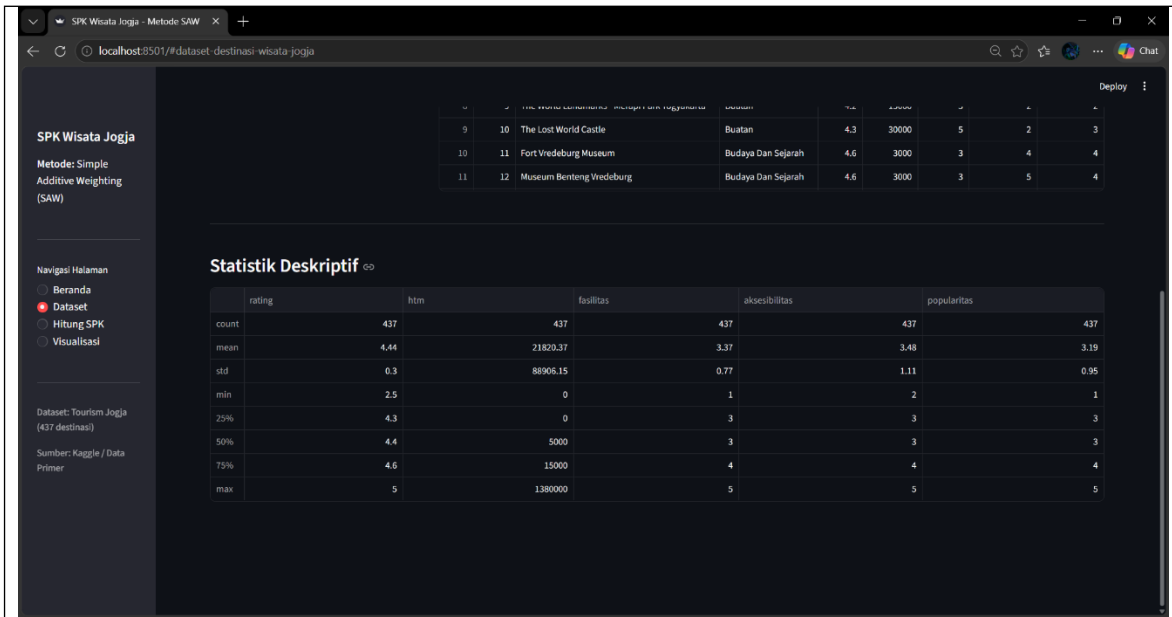
Minimum Rating

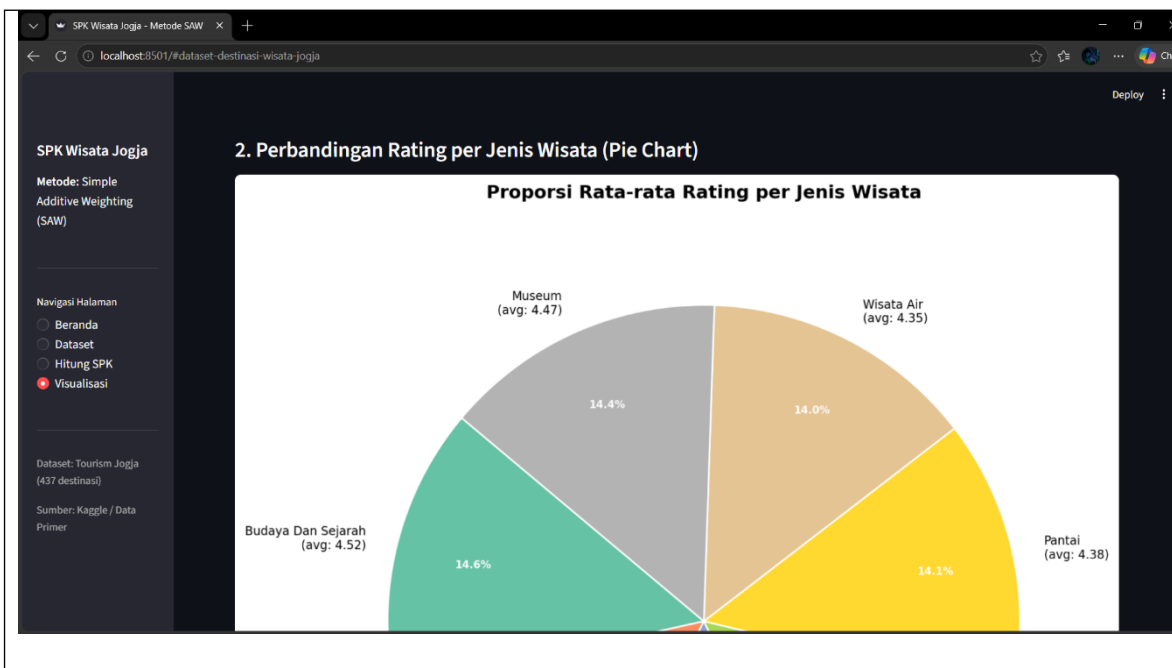
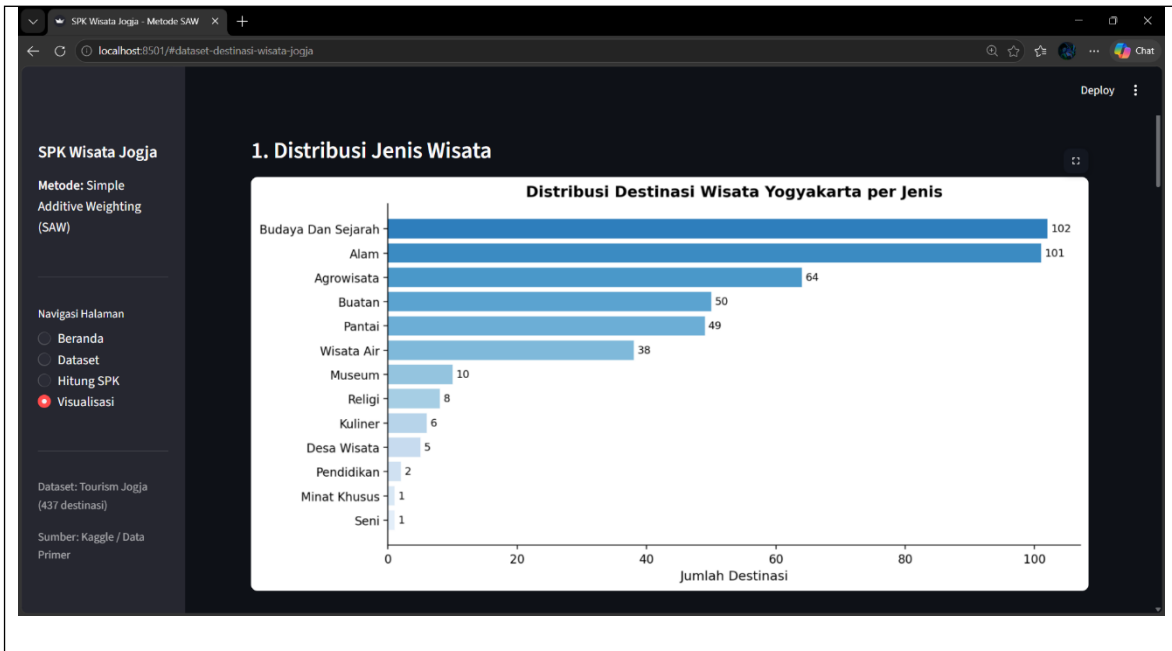
4.00

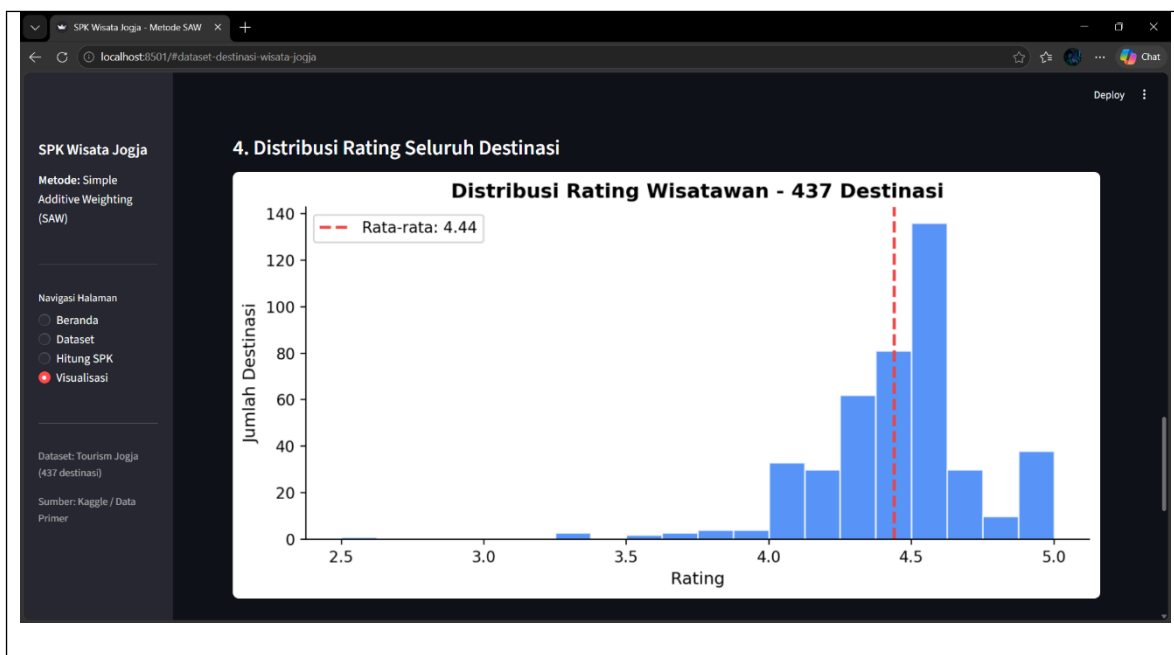
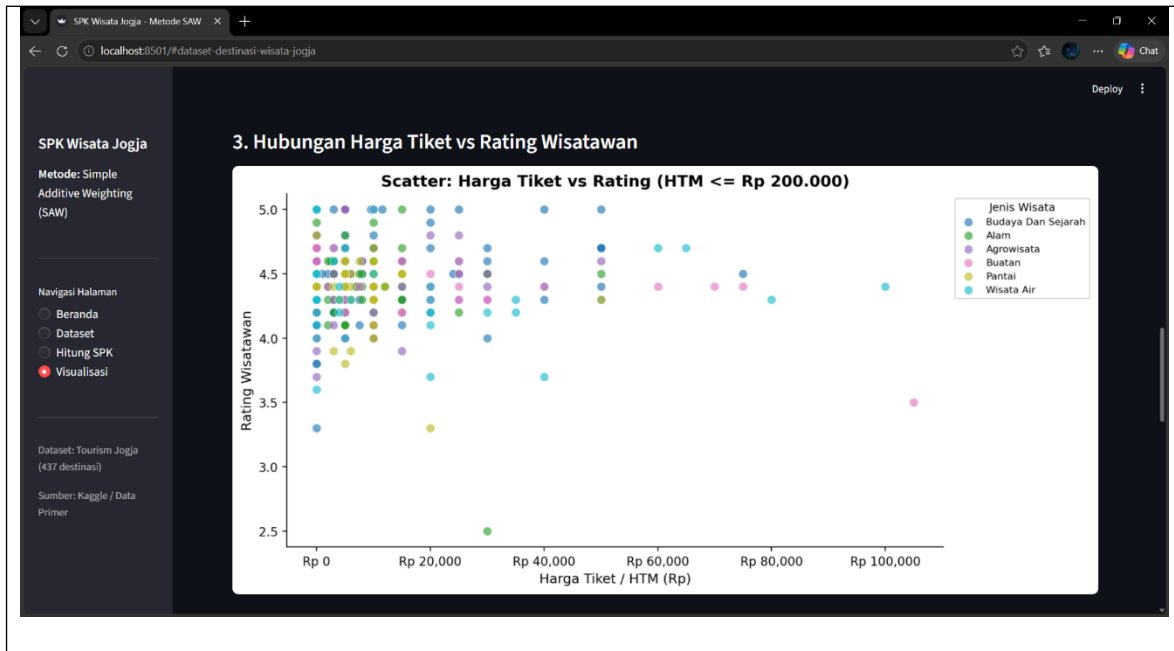
Menampilkan 420 destinasi sesuai filter

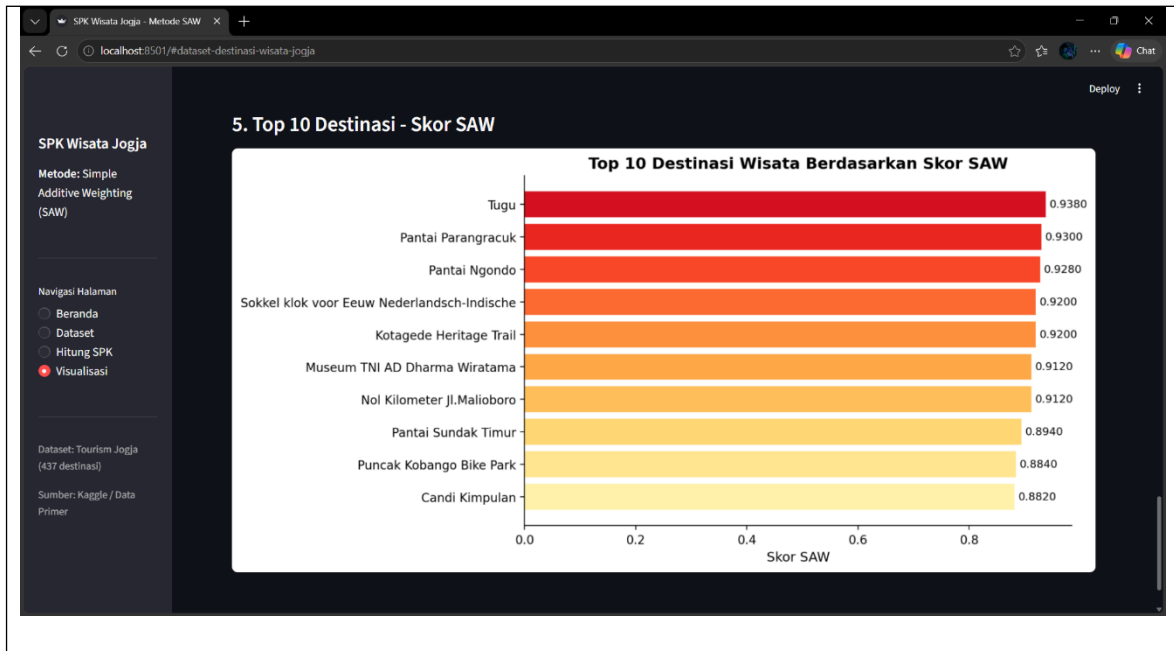
ID	Nama Destinasi	Jenis	Rating	HTM (Rp)	Fasilitas	Aksesibilitas	Popularitas	
0	1	Candi Borobudur	Budaya Dan Sejarah	4.7	50000	3	2	4
1	2	Candi Prambanan	Budaya Dan Sejarah	4.7	50000	3	5	4
2	3	Tebing Breksi	Alam	4.4	10000	3	4	3
3	4	Gembira Loka Zoo	Buatan	4.5	25000	5	3	3
4	5	The Palace of Yogyakarta (Keraton Yogyakarta)	Budaya Dan Sejarah	4.6	8000	3	3	4
5	6	Taman Sari	Budaya Dan Sejarah	4.6	5000	3	5	4
6	7	Hutan Pinus Mungunan Dlingo	Alam	4.6	3000	3	2	4
7	8	Jogja Bay	Wisata Air	4.4	100000	4	4	3
8	9	The World Landmarks - Merapi Park Yogyakarta	Buatan	4.2	15000	5	2	2
9	10	The Lost World Castle	Buatan	4.3	30000	5	2	3
10	11	Fort Vredeburg Museum	Budaya Dan Sejarah	4.6	3000	3	4	4
11	12	Museum Benteng Vredeburg	Budaya Dan Sejarah	4.6	3000	3	5	4

15









BAB III

JADWAL Pengerjaan dan Pembagian Tugas

3.1 Jadwal Pengerjaan

No	Kegiatan	April 2026				
		Minggu				
		1	2	3	4	dst
1	Penentuan Judul dan Studi Kasus	√				
2	Pengumpulan Dataset (Kaggle)		√			
3	Analisis Kriteria dan Pembobotan				√	
4	Pemodelan Sistem (Metode SAW)				√	
dst						

No	Kegiatan	Mei 2026				
		Minggu				
		1	2	3	4	dst
1	Pengembangan Aplikasi (Python & Streamlit)		√			
2	Uji Coba dan Visualisasi Data			√		
3	Penyusunan Laporan Akhir			√		
4						
dst						

3.2 Pembagian Tugas

No	Nama	NIM	Deskripsi Tugas
1	A Anshori	123240020	- Melakukan <i>preprocessing</i> dataset "Tourism Jogja" dari Kaggle - Mengimplementasikan logika perhitungan metode Simple Additive Weighting (SAW) ke dalam kode Python. - Menyusun skenario kasus dan mendefinisikan kriteria (C1-C5) beserta jenisnya (Benefit/Cost). - Menentukan atribut turunan seperti skor fasilitas, aksesibilitas, dan popularitas. - Menyusun BAB II (Pembahasan) bagian Perhitungan & Algoritma.
2	M Hasan R	123240005	- Membangun struktur utama aplikasi menggunakan framework Streamlit. - Merancang tampilan antarmuka (UI) pada halaman Beranda dan Dataset di Streamlit. - Mengembangkan bagian visualisasi data (grafik dan tabel peringkat). - Menyusun BAB I (Pendahuluan) termasuk Latar Belakang, Tujuan, dan Manfaat. - Melakukan integrasi dokumen dan finalisasi laporan proyek akhir.

BAB IV

KESIMPULAN DAN SARAN

4.1 Kesimpulan

Berdasarkan hasil perancangan dan implementasi Sistem Pendukung Keputusan (SPK) untuk rekomendasi destinasi wisata di Yogyakarta menggunakan metode *Simple Additive Weighting* (SAW), dapat diambil kesimpulan sebagai berikut:

1. Efektivitas Metode: Metode SAW berhasil diimplementasikan untuk memberikan peringkat destinasi wisata secara objektif berdasarkan lima kriteria utama, yaitu harga tiket (C1), rating (C2), jumlah ulasan (C3), fasilitas (C4), dan popularitas (C5).
2. Hasil Rekomendasi: Sistem mampu mengolah dataset pariwisata Jogja dan menghasilkan urutan prioritas yang akurat sesuai dengan bobot yang ditentukan. Hal ini membantu calon wisatawan dalam mengambil keputusan yang lebih cepat sesuai dengan preferensi budget dan kenyamanan mereka.
3. Fungsionalitas Aplikasi: Aplikasi berbasis Streamlit yang dikembangkan telah berfungsi dengan baik, mulai dari proses preprocessing data, normalisasi matriks, hingga visualisasi hasil peringkat dalam bentuk grafik yang mudah dipahami oleh pengguna.

4.2 Saran

Untuk pengembangan sistem yang lebih baik di masa mendatang, terdapat beberapa saran yang dapat dilakukan:

1. Penambahan Kriteria: Menambahkan kriteria yang lebih spesifik seperti jarak dari lokasi pengguna (menggunakan API Google Maps) atau kategori jenis wisata (wisata alam, edukasi, atau kuliner) agar hasil lebih personal.
2. Integrasi Metode Lain: Melakukan komparasi hasil dengan metode SPK lainnya seperti TOPSIS atau WP (Weighted Product) untuk memvalidasi tingkat akurasi rekomendasi.
3. Pembaruan Data Otomatis: Mengintegrasikan sistem dengan web scraping agar data rating dan jumlah ulasan selalu terbaru secara real-time sesuai dengan kondisi di lapangan.
4. Fitur User Profile: Menambahkan fitur login agar pengguna dapat menyimpan hasil pencarian atau memberikan penilaian pribadi terhadap destinasi yang telah dikunjungi.
5. Penerapan Model Hybrid: Disarankan untuk mengembangkan sistem menggunakan model Hybrid Fuzzy-SAW, di mana logika Fuzzy digunakan untuk memproses pembobotan kriteria yang bersifat subjektif, sementara metode SAW digunakan untuk kalkulasi perankingan akhir, guna menghasilkan rekomendasi yang lebih akurat dan adaptif.

DAFTAR PUSTAKA

Rizqiawan, F. (2020). *Dataset Rekomendasi Wisata Jogja*. Kaggle.
<https://www.kaggle.com/datasets/farisrizqiawan/dataset-rekomendasi-wisata-jogja>

- Jangan pakai angka (1, 2, 3), tapi disusun berdasarkan Abjad (A-Z) dari nama belakang pengarang.
- Nama belakang/keluarga ditulis duluan, baru diikuti inisial atau singkatan nama depannya.
- Judul buku atau jurnal tidak boleh disingkat.
- Judul buku atau nama jurnal biasanya ditulis miring (*italic*).
- Baris kedua dan seterusnya dalam satu referensi harus menjorok ke dalam (sekitar 1,27 cm atau 1 tab)